

Real-Time Examples of Dependency Inversion Principle in C#

In this article, I will discuss **Multiple Real-Time Examples of the Dependency Inversion Principle (DIP) in C#**. Please read our previous article discussing the basic concepts of the [Dependency Inversion Principle \(DIP\) in C#](#). At the end of this article, you will understand the following Real-time Examples using the Dependency Inversion Principle (DIP) in C#.

1. [Message Logger System](#)
2. [Payment Processing System](#)
3. [Sends Email Notifications to Users](#)
4. [Music Player System](#)
5. [Data Storage](#)
6. [Vehicle Maintenance System](#)
7. [User Authentication](#)
8. [Placing Order in an E-commerce Application](#)

What is the Dependency Inversion Principle (DIP)?

When designing object-oriented systems, it is important to follow the Dependency Inversion Principle (DIP), which is part of the SOLID principles. The main idea of DIP is to use abstractions instead of concrete implementations. By separating high-level modules with complex features from low-level modules that provide basic operations, DIP promotes the use of an abstraction layer to reduce coupling between the modules.

How to use the Dependency Inversion Principle in C#?

Using the Dependency Inversion Principle (DIP) in C# involves designing classes and modules to promote loose coupling and dependency inversion. Here's a step-by-step guide on how to use the Dependency Inversion Principle effectively in C#:

- **Identify High-Level and Low-Level Modules:** Identify your application's high-level and low-level modules, with the high-level modules containing business logic and the low-level modules containing implementation details.
- **Define Abstractions:** Create interfaces or abstract classes to define the behavior or contract that high-level modules need.
- **Implement Abstractions:** Implement the interfaces or abstract classes in the low-level modules to fulfill the contracts defined by the abstractions with concrete details.
- **Invert Dependencies:** Make high-level modules depend on abstractions (interfaces or abstract classes) instead of low-level module implementations.
- **Use Dependency Injection:** Implement dependency injection to provide instances of implementations to high-level modules. We can control the high-level module behavior without modifying their code.
- **IoC Containers (Optional):** Consider using an IoC container for dependency injection and object lifetime management. IoC containers simplify the process of injecting dependencies and managing object lifetimes.
- **Configure Dependencies:** Configure your application to inject the appropriate implementations into high-level modules through constructors, properties, or methods.
- **Isolate Unit Testing:** When conducting unit testing, abstract implementations are replaced with mock implementations to isolate high-level modules from the actual implementations.

Real-Time Example of Dependency Inversion Principle in C#: Message Logger System

Suppose we are building a system where we want to log messages. Initially, we might have a logging mechanism that writes messages to a console. However, in the future, we may want to switch to logging messages in a file or sending them over the network.

Without DIP:

Let us first see how we can implement the above example without following the Dependency Inversion Principle (DIP):

```
using System;
namespace DIPDemo
{
    public class ConsoleLogger
    {
        public void LogMessage(string message)
        {
            Console.WriteLine(message);
        }
    }
    public class NotificationService
    {
        private ConsoleLogger _logger = new ConsoleLogger();
        public void Notify(string message)
        {
            // ... some notification logic ...
            _logger.LogMessage(message);
        }
    }
}
```

The problem here is that NotificationService is directly dependent on ConsoleLogger. What if we want to change our logging mechanism?

With DIP:

Let us see how we can rewrite the above example following the Dependency Inversion Principle (DIP). The following example code is self-explained, so please go through the comment lines for a better understanding.

```
using System;
using System.IO;
namespace DIPDemo
{
    //Interface for logging
    public interface ILogger
    {
        void LogMessage(string message);
    }
}
```

```

}
//Concrete Loggers
public class ConsoleLogger : ILogger
{
    public void LogMessage(string message)
    {
        Console.WriteLine(message);
    }
}
public class FileLogger : ILogger
{
    private string _filePath;
    public FileLogger(string filePath)
    {
        _filePath = filePath;
    }
    public void LogMessage(string message)
    {
        // Just a simple example. In a real-world scenario, proper exception
and file IO management is needed.
        File.AppendAllText(_filePath, message);
    }
}
//Now, our NotificationService should depend on the abstraction
public class NotificationService
{
    private ILogger _logger;
    public NotificationService(ILogger logger)
    {
        _logger = logger;
    }
    public void Notify(string message)
    {
        // ... some notification logic ...
        _logger.LogMessage(message);
    }
}

//Testing the Dependency Inversion Principle
public class Program
{
    public static void Main()
    {
        //Now, when initializing the NotificationService,
        //we can decide which logger to use:
        var consoleLogger = new ConsoleLogger();
        var notificationService1 = new NotificationService(consoleLogger);
        var fileLogger = new FileLogger("path_to_log_file.txt");
        var notificationService2 = new NotificationService(fileLogger);
        Console.ReadKey();
    }
}

```

```
}  
}
```

By applying the Dependency Inversion Principle (DIP), we have decoupled the NotificationService from the concrete implementation of the logger. This makes the system more flexible and open for extension.

Real-Time Example of Dependency Inversion Principle in C#: Payment Processing System

Let's consider an example closer to a real-world scenario: a system that processes payments. You might have multiple payment methods (Credit Card, PayPal, etc.), and the system should be flexible enough to handle any new payment methods in the future.

Without DIP:

Let us first see how we can implement the above example without following the Dependency Inversion Principle (DIP). Here, the PaymentProcessor directly depends on concrete payment method classes:

```
using System;  
namespace DIPDemo  
{  
    public class CreditCard  
    {  
        public void ProcessPayment(decimal amount)  
        {  
            Console.WriteLine($"Processing credit card payment of {amount}");  
        }  
    }  
    public class PaymentProcessor  
    {  
        public void ExecutePayment(decimal amount)  
        {  
            var creditCard = new CreditCard();  
            creditCard.ProcessPayment(amount);  
        }  
    }  
}
```

This design tightly couples the PaymentProcessor to the CreditCard class. What if we want to add PayPal or any other payment method in the future?

With DIP:

Let us see how we can rewrite the above example following the Dependency Inversion Principle (DIP):

```
using System;  
namespace DIPDemo  
{  
    //Interface for Payment
```

```

public interface IPaymentMethod
{
    void ProcessPayment(decimal amount);
}

//Concrete Implementations
public class CreditCard : IPaymentMethod
{
    public void ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Processing credit card payment of {amount}");
    }
}
public class PayPal : IPaymentMethod
{
    public void ProcessPayment(decimal amount)
    {
        Console.WriteLine($"Processing PayPal payment of {amount}");
    }
}
//Our PaymentProcessor class will now depend on the abstraction
public class PaymentProcessor
{
    private readonly IPaymentMethod _paymentMethod;
    public PaymentProcessor(IPaymentMethod paymentMethod)
    {
        _paymentMethod = paymentMethod;
    }
    public void ExecutePayment(decimal amount)
    {
        _paymentMethod.ProcessPayment(amount);
    }
}

//Testing the Dependency Inversion Principle
public class Program
{
    public static void Main()
    {
        var creditCardPayment = new CreditCard();
        var paymentProcessor1 = new PaymentProcessor(creditCardPayment);
        paymentProcessor1.ExecutePayment(100m);
        var paypalPayment = new PayPal();
        var paymentProcessor2 = new PaymentProcessor(paypalPayment);
        paymentProcessor2.ExecutePayment(100m);

        Console.ReadKey();
    }
}

```

By decoupling the PaymentProcessor from specific implementations, it becomes simpler to add or modify payment methods without altering the PaymentProcessor class. This takes advantage of the Dependency Inversion Principle, which makes the system more flexible and maintainable. When you run the above code, you will get the following output.

```
Processing credit card payment of 100
Processing PayPal payment of 100
```

Real-Time Example of Dependency Inversion Principle in C#: Sends Email Notifications to Users

Let's understand another real-time scenario: an e-commerce system that sends email notifications to users. These notifications can be related to order updates, promotional offers, etc. Today, the system might use an SMTP protocol to send emails. Still, in the future, it might shift to using a third-party email service provider or even other methods of communication.

Without DIP:

Let us first see how we can implement the above example without following the Dependency Inversion Principle (DIP). Here, the OrderProcessor is directly dependent on a specific email-sending mechanism:

```
using System;
namespace DIPDemo
{
    public class Order
    {
        public string OrderId { get; set; }
        public decimal Amount { get; set; }
        public string CustomerEmail { get; set; }
    }
    public class SmtplibMailer
    {
        public void SendEmail(string emailAddress, string subject, string content)
        {
            // Logic to send email using SMTP
            Console.WriteLine($"Email sent to {emailAddress} using SMTP: {subject} {content}");
        }
    }
    public class OrderProcessor
    {
        private SmtplibMailer _mailer = new SmtplibMailer();
        public void ProcessOrder(Order order)
        {
            // Order processing logic...
            _mailer.SendEmail(order.CustomerEmail, "Order Processed", "Your order has been processed.");
        }
    }
}
```

This design ties the OrderProcessor to the SmtplibMailer. If we were to shift to a different email service or introduce SMS notifications, this would mean substantial changes.

With DIP:

Let us see how we can rewrite the above example following the Dependency Inversion Principle (DIP):

```
using System;
namespace DIPDemo
{
    //Define the Order model
    public class Order
    {
        public string OrderId { get; set; }
        public decimal Amount { get; set; }
        public string CustomerEmail { get; set; }
    }
    //Define the Interface
    public interface INotificationService
    {
        void Notify(string recipient, string subject, string message);
    }
    //Concrete implementations
    public class SmtplibMailer : INotificationService
    {
        public void Notify(string recipient, string subject, string message)
        {
            // Logic to send email using SMTP
            Console.WriteLine($"Email sent to {recipient} using SMTP: {subject}");
        }
    }
    public class ThirdPartyEmailService : INotificationService
    {
        public void Notify(string recipient, string subject, string message)
        {
            // Logic to send email using a third-party service
            Console.WriteLine($"Email sent to {recipient} using ThirdPartyEmailService: {subject}");
        }
    }
    //Now, the OrderProcessor will be
    public class OrderProcessor
    {
        private readonly INotificationService _notificationService;
        public OrderProcessor(INotificationService notificationService)
        {
            _notificationService = notificationService;
        }
        public void ProcessOrder(Order order)
        {
            // Logic to process the order
            // ...
            // Call the notification service
            _notificationService.Notify(order.CustomerEmail, order.OrderId, order.Amount.ToString());
        }
    }
}
```

```

        // Order processing logic...
        _notificationService.Notify(order.CustomerEmail, "Order Processed'
order has been processed.");
    }
}

//Testing the Dependency Inversion Principle
public class Program
{
    public static void Main()
    {
        var smtpNotification = new SmtplibMailer();
        var orderProcessor1 = new OrderProcessor(smtpNotification);
        orderProcessor1.ProcessOrder(new Order { CustomerEmail =
"customer@example.com" });
        var thirdPartyNotification = new ThirdPartyEmailService();
        var orderProcessor2 = new OrderProcessor(thirdPartyNotification);
        orderProcessor2.ProcessOrder(new Order { CustomerEmail =
"customer@example.com" });

        Console.ReadKey();
    }
}

```

Thanks to the Dependency Inversion Principle, with this design, we can easily adapt to changing requirements by allowing the addition of new notification methods (e.g., SMS, push notifications) or switching between different email service providers.

Real-Time Example of Dependency Inversion Principle in C#: Music Player System

Imagine you have a music player that can play songs from various sources, such as local storage, online streaming services, or radio stations.

Without DIP:

Let us first see how we can implement the above example without following the Dependency Inversion Principle (DIP). Here, the MusicPlayer is directly dependent on a specific source, e.g., local storage:

```

using System;
namespace DIPDemo
{
    public class LocalStorage
    {
        public void PlayFromDevice(string trackName)
        {
            Console.WriteLine($"Playing {trackName} from local storage...");
        }
    }
    public class MusicPlayer

```

```

{
    private LocalStorage _storage = new LocalStorage();
    public void Play(string trackName)
    {
        _storage.PlayFromDevice(trackName);
    }
}
}

```

This design binds the MusicPlayer directly to LocalStorage. If we want the player to support other sources, we'd need to make significant modifications.

With DIP:

Let us see how we can rewrite the above example following the Dependency Inversion Principle (DIP)

```

using System;
namespace DIPDemo
{
    //Define the Interface
    public interface IMusicSource
    {
        void PlayTrack(string trackName);
    }
    //Concrete implementations
    public class LocalStorage : IMusicSource
    {
        public void PlayTrack(string trackName)
        {
            Console.WriteLine($"Playing {trackName} from local storage...");
        }
    }
    public class StreamingService : IMusicSource
    {
        public void PlayTrack(string trackName)
        {
            Console.WriteLine($"Streaming {trackName} from online service...");
        }
    }
    public class RadioStation : IMusicSource
    {
        public void PlayTrack(string stationName)
        {
            Console.WriteLine($"Tuning into {stationName} radio station...");
        }
    }
    //The MusicPlayer class now depends on the abstraction
    public class MusicPlayer
    {
        private readonly IMusicSource _source;
    }
}

```

```

    public MusicPlayer(IMusicSource source)
    {
        _source = source;
    }
    public void Play(string name)
    {
        _source.PlayTrack(name);
    }
}

//Testing the Dependency Inversion Principle
public class Program
{
    public static void Main()
    {
        var local = new LocalStorage();
        var playerFromLocal = new MusicPlayer(local);
        playerFromLocal.Play("SongA.mp3");
        var streaming = new StreamingService();
        var playerFromStreaming = new MusicPlayer(streaming);
        playerFromStreaming.Play("SongB");
        var radio = new RadioStation();
        var radioPlayer = new MusicPlayer(radio);
        radioPlayer.Play("Jazz FM");
        Console.ReadKey();
    }
}
}

```

By applying the Dependency Inversion Principle, the MusicPlayer is decoupled from specific music sources. This enables the integration of new sources or modifications to existing ones without changing the MusicPlayer class. Consequently, the system becomes more maintainable and can adjust to adaptable needs.

Real-Time Example of Dependency Inversion Principle in C#: Data Storage

Imagine you're building a system where users can save and retrieve their personal notes. Initially, these notes are saved to a database. Still, given the rapid technology changes, you want to ensure your system remains flexible, potentially allowing for storage in a cloud platform or even flat files.

Without DIP:

Let us first see how we can implement the above example without following the Dependency Inversion Principle (DIP).

```

using System;
namespace DIPDemo
{
    public class Database
    {

```

```

        public void Save(string data)
        {
            Console.WriteLine($"Saving note to database: {data}");
        }
        public string Retrieve(int id)
        {
            return $"Note {id} from database";
        }
    }
    public class NoteManager
    {
        private Database _db = new Database();
        public void SaveNote(string note)
        {
            _db.Save(note);
        }
        public string GetNote(int id)
        {
            return _db.Retrieve(id);
        }
    }
}

```

This design directly couples the NoteManager to the Database, limiting flexibility.

With DIP:

Let us see how we can rewrite the above example following the Dependency Inversion Principle (DIP)

```

using System;
namespace DIPDemo
{
    //Define the Interface
    public interface IDataStore
    {
        void Save(string data);
        string Retrieve(int id);
    }
    //Concrete implementations
    public class Database : IDataStore
    {
        public void Save(string data)
        {
            Console.WriteLine($"Saving note to database: {data}");
        }
        public string Retrieve(int id)
        {
            return $"Note {id} from database";
        }
    }
}

```

```

public class CloudStorage : IDataStore
{
    public void Save(string data)
    {
        Console.WriteLine($"Saving note to cloud: {data}");
    }
    public string Retrieve(int id)
    {
        return $"Note {id} from cloud";
    }
}
//The NoteManager now relies on an abstraction
public class NoteManager
{
    private readonly IDataStore _dataStore;
    public NoteManager(IDataStore dataStore)
    {
        _dataStore = dataStore;
    }
    public void SaveNote(string note)
    {
        _dataStore.Save(note);
    }
    public string GetNote(int id)
    {
        return _dataStore.Retrieve(id);
    }
}

//Testing the Dependency Inversion Principle
public class Program
{
    public static void Main()
    {
        var dbStore = new Database();
        var noteManagerWithDB = new NoteManager(dbStore);
        noteManagerWithDB.SaveNote("My first note.");
        Console.WriteLine(noteManagerWithDB.GetNote(1));
        var cloudStore = new CloudStorage();
        var noteManagerWithCloud = new NoteManager(cloudStore);
        noteManagerWithCloud.SaveNote("My cloud note.");
        Console.WriteLine(noteManagerWithCloud.GetNote(2));

        Console.ReadKey();
    }
}

```

When we follow the Dependency Inversion Principle in this design, it becomes easy to add or modify data storage mechanisms. The NoteManager remains consistent and doesn't require any changes, regardless of where the notes are stored. When you run the above code, you will get the following output.

```
Saving note to database: My first note.  
Note 1 from database  
Saving note to cloud: My cloud note.  
Note 2 from cloud
```

Real-Time Example of Dependency Inversion Principle in C#: Vehicle Maintenance System

Imagine you have a system that schedules and performs maintenance tasks on various types of vehicles, like cars, bikes, and trucks. Depending on the vehicle type, maintenance operations can be different.

Without DIP:

Let us first see how we can implement the above example without following the Dependency Inversion Principle (DIP). In this naive design, the MaintenanceService class might have hard dependencies on specific vehicle types:

```
using System;  
namespace DIPDemo  
{  
    public class Car  
    {  
        public void PerformCarMaintenance()  
        {  
            Console.WriteLine("Performing car maintenance...");  
        }  
    }  
    public class Bike  
    {  
        public void PerformBikeMaintenance()  
        {  
            Console.WriteLine("Performing bike maintenance...");  
        }  
    }  
    public class MaintenanceService  
    {  
        public void MaintainCar(Car car)  
        {  
            car.PerformCarMaintenance();  
        }  
        public void MaintainBike(Bike bike)  
        {  
            bike.PerformBikeMaintenance();  
        }  
    }  
}
```

The design above rigidly ties the MaintenanceService to specific vehicle implementations. If we want to add another vehicle type, say a Truck, the MaintenanceService needs modification.

With DIP:

Let us see how we can rewrite the above example following the Dependency Inversion Principle (DIP)

```
using System;
namespace DIPDemo
{
    //Define the Interface
    public interface IVehicle
    {
        void PerformMaintenance();
    }
    //Concrete implementations
    public class Car : IVehicle
    {
        public void PerformMaintenance()
        {
            Console.WriteLine("Performing car maintenance...");
        }
    }
    public class Bike : IVehicle
    {
        public void PerformMaintenance()
        {
            Console.WriteLine("Performing bike maintenance...");
        }
    }
    public class Truck : IVehicle
    {
        public void PerformMaintenance()
        {
            Console.WriteLine("Performing truck maintenance...");
        }
    }
    //The MaintenanceService class will now depend on the abstraction
    public class MaintenanceService
    {
        public void MaintainVehicle(IVehicle vehicle)
        {
            vehicle.PerformMaintenance();
        }
    }

    //Testing the Dependency Inversion Principle
    public class Program
    {
        public static void Main()
```

```

    {
        var car = new Car();
        var bike = new Bike();
        var truck = new Truck();
        var maintenanceService = new MaintenanceService();
        maintenanceService.MaintainVehicle(car);
        maintenanceService.MaintainVehicle(bike);
        maintenanceService.MaintainVehicle(truck);

        Console.ReadKey();
    }
}

```

By following the Dependency Inversion Principle, the MaintenanceService is not limited to a specific type of vehicle. This makes maintenance and expansion easier, even if there are changes in the maintenance process for an existing vehicle or a new type of vehicle is introduced. This separation improves the overall design's maintainability and scalability.

Real-Time Example of Dependency Inversion Principle in C#: User Authentication

Imagine you're building a system that allows users to authenticate. Initially, users authenticate using a username and password system stored in a local database. However, you anticipate that in the future, you may wish to allow users to authenticate using third-party services like Google, Facebook, or even biometrics.

Without DIP:

Let us first see how we can implement the above example without following the Dependency Inversion Principle (DIP). Here, the AuthenticationService is directly dependent on a specific authentication mechanism:

```

using System;
namespace DIPDemo
{
    public class DatabaseAuthenticator
    {
        public bool Authenticate(string username, string password)
        {
            // Logic to authenticate using a database
            Console.WriteLine($"Authenticating {username} using database.");
            return true;
        }
    }
    public class AuthenticationService
    {
        private DatabaseAuthenticator _authenticator = new DatabaseAuthenticator();
        public bool AuthenticateUser(string username, string password)
        {
            return _authenticator.Authenticate(username, password);
        }
    }
}

```

```
}  
}
```

This design tightly couples the AuthenticationService to the DatabaseAuthenticator.

With DIP:

Let us see how we can rewrite the above example following the Dependency Inversion Principle (DIP).

```
using System;  
namespace DIPDemo  
{  
    //Define the Interface  
    public interface IAuthenticator  
    {  
        bool Authenticate(string identifier, string credential);  
    }  
    //Concrete implementations  
    public class DatabaseAuthenticator : IAuthenticator  
    {  
        public bool Authenticate(string username, string password)  
        {  
            // Logic to authenticate using a database  
            Console.WriteLine($"Authenticating {username} using database.");  
            return true;  
        }  
    }  
    public class GoogleAuthenticator : IAuthenticator  
    {  
        public bool Authenticate(string token, string _)  
        {  
            // Logic to authenticate using Google  
            Console.WriteLine($"Authenticating using Google with token: {token}");  
            return true;  
        }  
    }  
    //The AuthenticationService class will now depend on the abstraction  
    public class AuthenticationService  
    {  
        private readonly IAuthenticator _authenticator;  
        public AuthenticationService(IAuthenticator authenticator)  
        {  
            _authenticator = authenticator;  
        }  
        public bool AuthenticateUser(string identifier, string credential)  
        {  
            return _authenticator.Authenticate(identifier, credential);  
        }  
    }  
}
```

```
//Testing the Dependency Inversion Principle
public class Program
{
    public static void Main()
    {
        var dbAuthenticator = new DatabaseAuthenticator();
        var authServiceWithDB = new AuthenticationService(dbAuthenticator);
        authServiceWithDB.AuthenticateUser("Alice", "password123");
        var googleAuthenticator = new GoogleAuthenticator();
        var authServiceWithGoogle = new AuthenticationService(googleAuthenticator);
        authServiceWithGoogle.AuthenticateUser("OAuthTokenHere", "");

        Console.ReadKey();
    }
}
}
```

According to the Dependency Inversion Principle, the AuthenticationService is decoupled from any specific authentication method. This allows for easy addition of new methods or changes to existing ones without affecting the central AuthenticationService.

Real-Time Example of Dependency Inversion Principle in C#: Placing Order in an E-commerce Application

Consider you are building an e-commerce system where customers can place orders. Upon order completion, the system needs to notify the customer. Initially, the system sends notifications via email. However, given the diverse needs of modern users, the system might need to provide SMS or even app-based notifications in the future.

Without DIP:

Let us first see how we can implement the above example without following the Dependency Inversion Principle (DIP). Here, the OrderProcessor is directly dependent on a specific notification mechanism:

```
using System;
namespace DIPDemo
{
    public class EmailNotifier
    {
        public void SendEmailNotification(string email, string message)
        {
            Console.WriteLine($"Sending email to {email} with message: {message}");
        }
    }
    public class OrderProcessor
    {
        private EmailNotifier _notifier = new EmailNotifier();
        public void CompleteOrder(string email, string orderDetails)
        {

```

```

        // Logic to process the order...
        _notifier.SendEmailNotification(email, "Your order has been comple
    }
}
}

```

This design tightly binds the OrderProcessor to the EmailNotifier. If we need to introduce SMS or app notifications, this approach will necessitate substantial changes.

With DIP:

Let us see how we can rewrite the above example following the Dependency Inversion Principle (DIP).

```

using System;
namespace DIPDemo
{
    //Define the Interface
    public interface INotifier
    {
        void Notify(string recipient, string message);
    }
    //Concrete implementations
    public class EmailNotifier : INotifier
    {
        public void Notify(string email, string message)
        {
            Console.WriteLine($"Sending email to {email} with message: {message}");
        }
    }
    public class SmsNotifier : INotifier
    {
        public void Notify(string phoneNumber, string message)
        {
            Console.WriteLine($"Sending SMS to {phoneNumber} with message: {message}");
        }
    }
    //The OrderProcessor class will now depend on the abstraction
    public class OrderProcessor
    {
        private readonly INotifier _notifier;
        public OrderProcessor(INotifier notifier)
        {
            _notifier = notifier;
        }
        public void CompleteOrder(string recipient, string orderDetails)
        {
            // Logic to process the order...
            _notifier.Notify(recipient, "Your order has been completed!");
        }
    }
}

```

```

//Testing the Dependency Inversion Principle
public class Program
{
    public static void Main()
    {
        var emailNotifier = new EmailNotifier();
        var orderProcessorWithEmail = new OrderProcessor(emailNotifier);
        orderProcessorWithEmail.CompleteOrder("user@example.com", "Order
details...");
        var smsNotifier = new SmsNotifier();
        var orderProcessorWithSms = new OrderProcessor(smsNotifier);
        orderProcessorWithSms.CompleteOrder("+1234567890", "Order details.

        Console.ReadKey();
    }
}

```

We have applied the Dependency Inversion Principle to decouple the OrderProcessor from specific notification implementations. This enables us to introduce new notification methods or modify existing ones without altering the core OrderProcessor, making our system more maintainable and flexible.

Here, in this article, I try to explain **Real-Time Examples of the Dependency Inversion Principle (DIP) in C#**. I hope you enjoy this Dependency Inversion Principle (DIP) Real-Time Examples using the C# article. Next, you should learn [Design Patterns using C#](#).